Interaktives Auswahlpanel

10× Taster/LED

„Drücken – Leuchten“

ESP32-S3 • Raspberry Pi 5 • Web-Dashboard

Keywords

CD4021BE • 74HC595 • PlatformIO • Python/aiohttp • WebSocket • HTML/JS • systemd

Stand: 20.01.2026
Version 2.5.3

Jan Unger
Tools: Claude Opus 4.5, ChatGPT 5.2

Inhaltsverzeichnis

Teil I: Theoretische Grundlagen **1**

1 Embedded-Grundlagen – Systematische Einführung **2**
1.1 Begriffsrahmen und Zielsetzung . . . 2
1.2 Hardware ist die Schnittstelle . . . 2
1.3 Spannungspegel – Die Sprache der Hardware . . . 2
1.4 Active-Low vs. Active-High . . . 3
1.5 Schieberegister – Pins multiplizieren . . . 4
1.6 SPI – Der Datenbus . . . 5
1.7 Timing – Determinismus ist Pflicht . . . 6
1.8 Debouncing – Mechanisches Prellen filtern . . . 7
1.9 One-Hot-Encoding – Die Invariante . . . 9
1.10 SSOT – Single Source of Truth . . . 9
1.11 Test-Strategie – Vertrauen durch Verifikation . . . 10
1.12 Zusammenfassung – Merkregeln . . . 11
1.13 Weiterführende Dokumentation . . . 11

2 Desktop → Embedded Mindset (10 Regeln) **13**
2.1 Beispiel: Selection Panel (One-Hot) – „Taster i wählt LED i“ . . . 13
2.2 Arduino-Loop vs. FreeRTOS (Prinzip, nicht Code) . . . 14
2.3 One-Hot-Ausgabe (Definition, byte-basiert) . . . 14
2.4 One-Hot-Checkliste (damit es wirklich immer stimmt) . . . 15

3 Embedded-Entwicklung – Engineering-Regeln **16**
3.1 Begriffsrahmen und Zielsetzung . . . 16
3.2 Hardware ist die API . . . 16
3.3 Determinismus vor Flexibilität . . . 17
3.4 Edges statt Pegel . . . 18
3.5 One-Hot ist Pflicht . . . 19
3.6 Skalierung durch Formeln . . . 20
3.7 Schichten einhalten . . . 21
3.8 Testen ist Pflicht . . . 22
3.9 Debugging-Strategie . . . 23
3.10 Häufige Fehler . . . 23
3.11 Checkliste vor Commit . . . 23
3.12 Weiterführende Dokumentation . . . 24

Teil II: Das Selection-Panel-Projekt **25**

4 Selection-Panel-Projekt – Gesamtübersicht **26**
4.1 Begriffsrahmen und Zielsetzung . . . 26
4.2 Systemarchitektur – Drei Ebenen . . . 26

4.3 Repository-Struktur . . . 27
4.4 SSOT – Single Source of Truth . . . 28
4.5 Firmware – Build, Flash, Test . . . 28
4.6 Dashboard – Entwicklung und Build . . . 29
4.7 Bridge – Raspberry Pi . . . 30
4.8 Deployment-Workflow . . . 31
4.9 Dokumentationsstruktur . . . 32
4.10 Typische Arbeitsabläufe . . . 32
4.11 Roadmap . . . 33
4.12 Siehe auch . . . 33

5 Serial-Protokoll – Robuste Frames mit COBS + CRC 34
5.1 Begriffsrahmen und Zielsetzung . . . 34
5.2 Das Grundproblem – Byte-Strom vs. Nachrichten . . . 34
5.3 COBS – Consistent Overhead Byte Stuffing . . . 35
5.4 CRC-16 – Integritätsprüfung . . . 35
5.5 Packet-Layout – Der Header . . . 36
5.6 Sende- und Empfangsprozess . . . 37
5.7 Message-Typen . . . 38
5.8 ACK/Retry-Mechanik . . . 39
5.9 Datenfluss – Vom Button zum Browser . . . 40
5.10 Größenrahmen und Limits . . . 41
5.11 Fehlerbehandlung . . . 41
5.12 Siehe auch . . . 42

6 React-Stack Grundlagen 43
6.1 Begriffsrahmen und Zielsetzung . . . 43
6.2 React – Komponenten statt DOM-Manipulation . . . 43
6.3 TypeScript – Fehler vor Laufzeit finden . . . 44
6.4 Vite – Build-Tool und Entwicklungsserver . . . 45
6.5 ESLint + Prettier – Code-Qualität automatisieren . . . 47
6.6 React Router – Client-seitige Navigation . . . 48
6.7 Tailwind CSS – Utility-First Styling . . . 49
6.8 Zusammenspiel im Selection-Panel-Projekt . . . 50
6.9 Siehe auch . . . 51

Anhang 52

A Glossar: Embedded-Entwicklung 53

Teil I
Theoretische Grundlagen

Mikrocontroller, Mindset, Entwicklungsprozess

Ziel

Dieses Kapitel vermittelt die fundamentalen Konzepte der Embedded-Entwicklung: Mikrocontroller-Architektur, ressourcenbewusstes Denken und professionelle Entwicklungspraktiken.

1 Embedded-Grundlagen – Systematische Einführung

1.1 Begriffsrahmen und Zielsetzung

Embedded-Systeme unterscheiden sich fundamental von Desktop-Software. Die „API“ ist physisch – Pins, Spannungspegel, Timing. Fehler führen nicht zu Exceptions, sondern zu falschen Signalen, zerstörter Hardware oder unerklärlichem Verhalten.

Begriff	Definition
GPIO	General Purpose Input/Output – frei konfigurierbarer Pin
HAL	Hardware Abstraction Layer – Schicht zwischen Software und Registern
Active-Low	Signal ist aktiv bei Pegel 0 (GND)
Active-High	Signal ist aktiv bei Pegel 1 (VCC)
Debouncing	Entprellung – Filterung mechanischer Prellvorgänge
SPI	Serial Peripheral Interface – synchroner Datenbus
Schieberegister	IC, das parallele und serielle Daten konvertiert

Tabelle 1: Begriffsrahmen Embedded-Grundlagen

Ziel: Du verstehst die physikalischen Grundlagen, die hinter dem Selection-Panel-Code stehen.

1.2 Hardware ist die Schnittstelle

Problem: In Desktop-Software rufst du Funktionen auf und bekommst Rückgabewerte. In Embedded-Systemen kommunizierst du mit der Außenwelt über elektrische Signale.

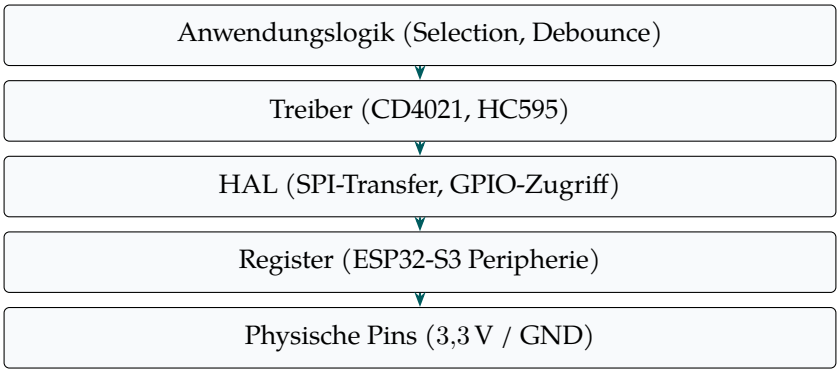


Abbildung 1: Schichten vom Code zur Hardware

Konsequenz

Ein Fehler auf jeder Ebene führt zu Fehlfunktion. Wenn der SPI-Modus falsch ist, liefert der CD4021 Müll – auch wenn dein Code syntaktisch korrekt ist.

1.3 Spannungspegel – Die Sprache der Hardware

Grundprinzip: Digitale Signale kennen zwei Zustände: HIGH und LOW.

Zustand	Spannung (3,3 V Logik)	Bedeutung
LOW	0 V – 0,8 V	Logisch 0
HIGH	2,0 V – 3,3 V	Logisch 1
Undefiniert	0,8 V – 2,0 V	Verboten!

Tabelle 2: Spannungspegel bei 3,3 V-Logik

Warum Pullups/Pulldowns?

Ein offener Pin (nicht verbunden) hat keinen definierten Pegel – er „schwebt“ und nimmt zufällige Werte an. Ein Pullup-Widerstand zieht den Pin auf HIGH, solange nichts anderes ihn auf LOW zieht.

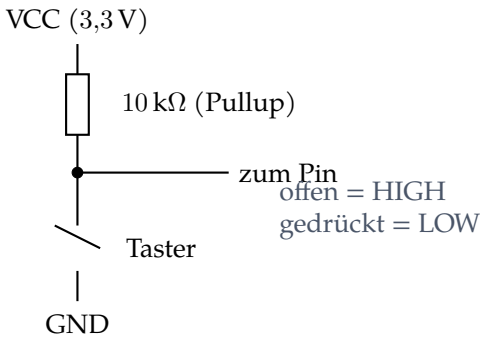


Abbildung 2: Pull-up-Beschaltung für Taster

Anwendung im Projekt

Die CD4021-Eingänge haben externe Pullups. Im Ruhezustand liest du HIGH (1), bei gedrücktem Taster LOW (0) – das ist Active-Low.

1.4 Active-Low vs. Active-High

Problem: Warum ist ein gedrückter Taster „0“ und eine leuchtende LED „1“? Das scheint unintuitiv.

Typ	Aktiv bei	Inaktiv bei	Typischer Einsatz
Active-Low	0 (GND)	1 (VCC)	Taster mit Pullup, Reset-Signale
Active-High	1 (VCC)	0 (GND)	LEDs, Enable-Signale

Tabelle 3: Active-Low vs. Active-High

Warum Active-Low für Taster?

Mit Pullup ist der Ruhezustand definiert (HIGH). Der Taster muss nur nach GND schalten – das ist schaltungstechnisch einfacher und robuster gegen Störungen.

Warum Active-High für LEDs?

LEDs leuchten, wenn Strom fließt. Bei Active-High: Pin HIGH → Strom fließt durch LED nach GND → LED leuchtet.

$$\texttt{Pin (HIGH) -- 330}\Omega\texttt{ -- LED -- GND}$$
$$3{,}3\,\text{V} \qquad \sim 2\,\text{V} \qquad \sim 0\,\text{V}$$

Konsequenz im Code:

```
// Button lesen (Active-Low)
bool isPressed = (raw & mask) == 0;  // Gedrueckt wenn Bit = 0

// LED setzen (Active-High)
if (ledOn) {
    ledState |= mask;   // Bit = 1 -> LED an
} else {
    ledState &= ~mask;  // Bit = 0 -> LED aus
}
```

1.5 Schieberegister – Pins multiplizieren

Problem: Der ESP32-S3 hat begrenzte GPIO-Pins. Für 100 Buttons und 100 LEDs bräuchtest du 200 Pins – die gibt es nicht.

Lösung: Schieberegister wandeln zwischen parallelen und seriellen Daten um.

CD4021 – Parallel zu Seriell (Buttons lesen)

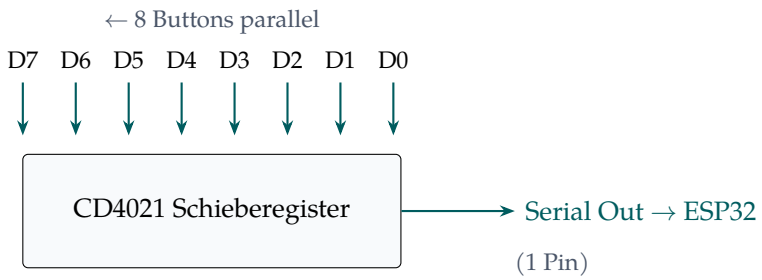


Abbildung 3: CD4021: Parallel-zu-Seriell-Wandlung

Ablauf:

1. **PS/LOAD = HIGH:** Parallele Eingänge werden ins Register übernommen
2. **PS/LOAD = LOW:** Register ist bereit zum Auslesen
3. **8× CLK-Pulse:** Bits werden nacheinander auf Serial Out geschoben

Timing:

Phase	PS/LOAD	CLK	Dauer
Load	HIGH	–	1 µs
Shift	LOW	↑↓ × 8	∼16 µs

Tabelle 4: CD4021-Timing

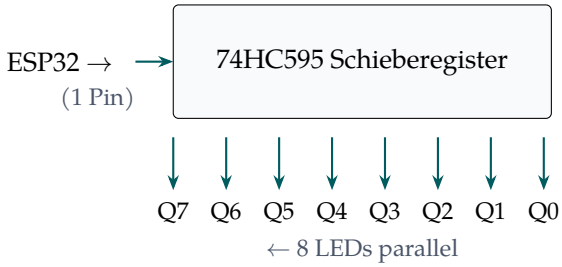


Abbildung 4: 74HC595: Seriell-zu-Parallel-Wandlung

74HC595 – Seriell zu Parallel (LEDs schreiben)

Ablauf:

1. **8× SRCLK-Pulse:** Bits werden ins Schieberegister geschoben
2. **LATCH-Puls (RCK):** Daten werden an die Ausgänge übernommen

Warum der LATCH?

Ohne Latch würden die LEDs während des Schiebens flackern. Der Latch hält die alten Werte, bis alle neuen Bits da sind.

Kaskadierung – Mehr als 8 Kanäle

Mehrere Schieberegister werden in Reihe geschaltet:

```
CD4021 (IC1) --> CD4021 (IC0) --> ESP32
   Buttons 9-16     Buttons 1-8      MISO

ESP32 --> HC595 (IC0) --> HC595 (IC1)
   MOSI     LEDs 1-8        LEDs 9-16
```

Konsequenz

Mit 13 CD4021 und 13 HC595 steuerst du 100 Buttons und 100 LEDs über nur 5 Pins (SCK, MISO, MOSI, PS, LATCH).

1.6 SPI – Der Datenbus

Problem: Wie überträgst du Daten zwischen ESP32 und Schieberegistern synchron und zuverlässig?

Lösung: SPI (Serial Peripheral Interface) – ein synchroner Bus mit Takt.

Signale

Signal	Richtung	Funktion
SCK	Master → Slave	Taktsignal (synchronisiert Übertragung)
MOSI	Master → Slave	Master Out, Slave In (Daten zum Slave)
MISO	Slave → Master	Master In, Slave Out (Daten vom Slave)
CS/SS	Master → Slave	Chip Select (welcher Slave ist aktiv)

Tabelle 5: SPI-Signale

SPI-Modi

Problem: Wann genau wird ein Bit gelesen – bei steigender oder fallender Flanke? Das hängt vom SPI-Modus ab.

Mode	CPOL	CPHA	Clock Idle	Sample bei
0	0	0	LOW	Rising Edge
1	0	1	LOW	Falling Edge
2	1	0	HIGH	Falling Edge
3	1	1	HIGH	Rising Edge

Tabelle 6: SPI-Modi

Im Projekt:

IC	Mode	Frequenz	Begründung
CD4021	1	500 kHz	Konservativ für Breadboard
74HC595	0	1 MHz	Standard

Tabelle 7: SPI-Konfiguration im Projekt

Konsequenz

Wenn du den falschen Modus verwendest, liest du Müll oder die Daten sind um ein Bit verschoben.

1.7 Timing – Determinismus ist Pflicht

Problem: Embedded-Software muss in definierten Zeitfenstern reagieren. Wenn du eine Deadline verpasst, funktioniert das System nicht.

Beispiel – Button-Scan:

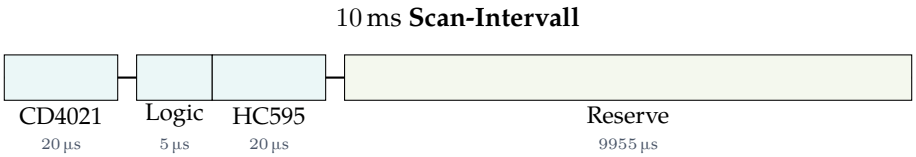


Abbildung 5: Timing-Budget eines Scan-Zyklus

Operation	Dauer	Anteil
CD4021 read (2 Bytes)	~20 µs	0,2 %
Debounce + Selection	~5 µs	0,05 %
HC595 write (2 Bytes)	~20 µs	0,2 %
Summe	~45 µs	~0,5 %
Reserve	~9955 µs	99,5 %

Tabelle 8: Timing-Budget

Budget bei 10 ms Intervall:

Was zerstört Timing?

Gut	Schlecht
`vTaskDelayUntil()` (FreeRTOS)	`delay()` im Hot-Path
Logging bei Zustandsänderung	Logging pro Zyklus
Kurze SPI-Transfers	Blockierende UART-Ausgabe

Tabelle 9: Timing-Killer vermeiden

Rechnung – UART-Overhead:

Bei 115 200 Baud dauert ein Zeichen $\frac{10\,\text{Bits}}{115200\,\text{Baud}} \approx 87\,\mu\text{s}$. Eine Debug-Zeile mit 50 Zeichen: $50 \times 87\,\mu\text{s} = 4{,}35\,\text{ms}$ – fast die Hälfte des Scan-Intervalls!

Konsequenz

Debug-Ausgaben nur bei Events (Zustandsänderungen), nie pro Scan-Zyklus.

1.8 Debouncing – Mechanisches Prellen filtern

Problem: Mechanische Taster prellen. Ein einzelner Druck erzeugt mehrere Flanken in wenigen Millisekunden.

```
Ideal:     ----+          +----
               |          |
               +----------+
                 gedrueckt
```

```
Real:      ----+ ++ ++    +++----
               | || ||    |||
               +-++-++----++-
                   Prellen
```

Messwerte:

Parameter	Typischer Wert
Prellzeit	5–50 ms
Prellvorgänge	3–20 Flanken

Tabelle 10: Typische Prellwerte

Lösung – Software-Debounce:

```c
typedef struct {
    uint8_t stable;     // Letzter stabiler Zustand
    uint8_t counter;    // Zaehler fuer aufeinanderfolgende gleiche Samples
} DebounceState;

bool debounce_update(DebounceState* state, uint8_t current, uint8_t threshold) {
    if (current == state->stable) {
        state->counter = 0;  // Zustand bestaetigt, Reset
        return false;
    }

    state->counter++;
    if (state->counter >= threshold) {
        // Neuer Zustand ist stabil
        bool wasLow = (state->stable == 0);
        bool isLow  = (current == 0);
        state->stable = current;
        state->counter = 0;

        // Press-Edge: Active-Low, war HIGH, ist jetzt LOW
        return (wasLow == false && isLow == true);
    }
    return false;  // Noch nicht stabil
}
```

Berechnung des Threshold:

Bei 10 ms Scan-Intervall und gewünschter 50 ms Debounce-Zeit:

$$\text{threshold} = \frac{50\,\text{ms}}{10\,\text{ms}} = 5$$

Konsequenz

Die Logik sieht nur stabile Zustände und saubere Flanken, kein Prellen.

1.9 One-Hot-Encoding – Die Invariante

Problem: Bei einem Selection Panel soll immer genau eine Auswahl aktiv sein (oder keine). Wie stellst du das sicher?

One-Hot-Prinzip:

```
selectedId = 0:  00000000 00000000  (nichts ausgewaehlt)
selectedId = 1:  10000000 00000000  (ID 1 aktiv)
selectedId = 2:  01000000 00000000  (ID 2 aktiv)
selectedId = 9:  00000000 10000000  (ID 9 aktiv)
```

Invariante: Zu jedem Zeitpunkt ist höchstens ein Bit gesetzt.

Implementierung:

```
void setOneHot(uint8_t selectedId, uint8_t* ledState, size_t bytes) {
    // 1. ALLE Bits loeschen (garantiert sauberen Zustand)
    memset(ledState, 0x00, bytes);

    // 2. Genau EIN Bit setzen (wenn ID > 0)
    if (selectedId >= 1 && selectedId <= LED_COUNT) {
        uint8_t byteIdx = (selectedId - 1) / 8;
        uint8_t bitPos  = 7 - ((selectedId - 1) % 8);  // MSB-first
        ledState[byteIdx] |= (1u << bitPos);
    }

    // 3. Ungueltige Bits im letzten Byte maskieren
    ledState[bytes - 1] &= LED_LAST_MASK;
}
```

Warum `memset` zuerst?

Andere Ansätze wie „altes Bit löschen, neues setzen“ können bei Bugs oder Race Conditions zu Zuständen mit zwei gesetzten Bits führen. Mit `memset` + einem `|=` ist die Invariante garantiert.

1.10 SSOT – Single Source of Truth

Problem: Wenn Pin-Definitionen in drei Dateien stehen und du eine änderst, sind die anderen falsch.

Regel: Jede Information hat genau eine autoritative Quelle.

Im Projekt:

Beispiel – `io_counts.h`:

Aspekt	SSOT-Datei
Pin-Zuordnung	`firmware/include/hw/board_pins.h`
Kanal-Anzahl	`firmware/include/hw/io_counts.h`
SPI-Parameter	`firmware/include/hw/spi_params.h`

Tabelle 11: SSOT-Dateien im Projekt

```
#define BTN_COUNT  10
#define LED_COUNT  10

// Abgeleitete Werte (automatisch korrekt bei Aenderung von COUNT)
#define BTN_BYTES  ((BTN_COUNT + 7) / 8)
#define LED_BYTES  ((LED_COUNT + 7) / 8)

// Maske fuer ungueltige Bits im letzten Byte
#define LED_LAST_MASK  0xC0  // Bei 10 LEDs: nur Bit 7, 6 gueltig
```

Konsequenz

Änderung von 10 auf 100 Kanäle erfordert nur zwei Zeilen in einer Datei.

1.11 Test-Strategie – Vertrauen durch Verifikation

Problem: Embedded-Bugs sind schwer zu debuggen. Du siehst nicht, was auf der Leitung passiert.

Lösung: Zweistufige Test-Strategie.

Unit-Tests (Logik isoliert)

```
pio test -d firmware -e seeed_xiao_esp32s3_test -v
```

Was wird getestet:

- Debounce-Algorithmus (filtert Prellen korrekt?)
- Selection-Logik (One-Hot-Invariante garantiert?)
- Bit-Mapping (ID → Byte/Bit korrekt?)

Vorteil: Läuft in Sekunden, keine Hardware nötig.

E2E-Tests (Hardware verifiziert)

```
pio test -d firmware -e seeed_xiao_esp32s3_e2e -f test_e2e_hw -v
```

Was wird getestet:

- SPI-Timing (Daten kommen korrekt an?)
- Pin-Zuordnung (Button 1 → Bit 7 von Byte 0?)
- LED-Mapping (selectedId=5 → LED 5 leuchtet?)

Ablauf (interaktiv):

1. Idle-Test: Nichts drücken, RAW sollte 0xFF sein
2. Button 1 drücken: RAW Bit 7 = 0?
3. Button 10 drücken: RAW Byte 1, Bit 6 = 0?
4. Walking-One: LEDs durchlaufen, visuell prüfen

Konsequenz

Nach Änderungen an SSOT-Dateien oder Verkabelung müssen beide Test-Stufen grün sein.

1.12 Zusammenfassung – Merkregeln

Spannungspegel

Zustand	3,3 V Logik
LOW	0 V – 0,8 V
HIGH	2,0 V – 3,3 V

Tabelle 12: Spannungspegel-Referenz

Active-Low/High

Komponente	Typ	Aktiv =
Buttons (mit Pullup)	Active-Low	0
LEDs	Active-High	1

Tabelle 13: Active-Low/High im Projekt

SPI-Modi (im Projekt)

Timing-Budget

1.13 Weiterführende Dokumentation

`firmware/docs/wiring.md` Hardware-Verdrahtung

IC	Mode	CLK Idle	Sample bei
CD4021	1	LOW	Falling Edge
74HC595	0	LOW	Rising Edge

Tabelle 14: SPI-Modi-Referenz

Scan-Intervall	IO-Operationen	Reserve
10 000 µs	∼45 µs	∼9955 µs

Tabelle 15: Timing-Budget-Referenz

`firmware/docs/architecture.md` Software-Architektur

`docs/grundlagen-bit-operationen.md` Bit-Manipulation

`docs/bit-mapping-schieberegister.md` ID-zu-Bit-Zuordnung

2 Desktop → Embedded Mindset (10 Regeln)

1. **Hardware ist die API**
   Pins, Busse, Register sind „die Schnittstelle“. OS-Abstraktionen fehlen oft.

2. **Worst-Case zählt, nicht der Durchschnitt**
   Plane Deadlines/Latenzen, nicht „läuft meistens“.

3. **Kein Blockieren im kritischen Pfad**
   Keine langen ISR, keine Busy-Waits, kein `delay()` in Logik/Tasks.

4. **ISR = minimal**
   ISR setzt Flag/Queue + Timestamp. Verarbeitung im Task/Mainloop.

5. **Speicher ist ein Budget**
   Stack/RAM sind eng. Große Daten statisch oder in Pools. Rekursion vermeiden.

6. **Nebenläufigkeit ist überall**
   ISR ↔ Task ↔ Main: Ownership und Synchronisation (Queue/Atomics) festlegen.

7. **C++ Feature-Set bewusst wählen**
   `constexpr`, `enum class`, kleine POD-Structs: ja. Exceptions/RTTI/iostream: meist nein.

8. **I/O entkoppeln**
   `read_inputs()` → `logic_update()` → `write_outputs()` (testbar, deterministisch).

9. **Fehler sind Normalfall**
   Timeouts, Brownout, Disconnect: Recovery + Watchdog einplanen.

10. **Observability ist knapp**
    Logging sparsam, Counters, Debug-Pins/Tracing. Debugbarkeit mitdesignen.

2.1 Beispiel: Selection Panel (One-Hot) – „Taster i wählt LED i“

Ziel: 10 Taster, 10 LEDs. Bei Tastendruck wird **genau eine** LED aktiv (One-Hot). Ein neuer Tastendruck schaltet **sofort** auf die neue Auswahl um.

2.1.1 Konventionen (wichtig, konsistent)

- **Buttons sind active-low (Hardware):** `raw_bit = 0` bedeutet „gedrückt“ (z. B. CD4021).
- **Logik arbeitet normalisiert:** `pressed = !raw_bit` → dann gilt in der Logik: `1 = gedrückt`.
- **Auswahl wird als ID geführt:**
  `selected_id` $\in \{0..10\}$
  `0` = keine Auswahl, `1..10` = LED/Taster-ID

2.1.2 Datenfluss (Architekturregel)

```
read_inputs_raw() -> normalize_active_low() -> debounce_update()
                  -> selection_update() -> write_outputs_onehot()
```

Tabelle 16: Datenfluss-Funktionen im Detail

Funktion	Beschreibung
`read_inputs_raw()`	Liest Bits aus dem Schieberegister (z. B. 2 Bytes für 10 Taster)
`normalize_active_low()`	Invertiert Bits → Logik-Semantik `1 = pressed`
`debounce_update()`	Macht aus prellenden Rohwerten stabile Zustände
`selection_update()`	Setzt `selected_id` bei neuem Press-Event
`write_outputs_onehot()`	Setzt genau **ein** LED-Bit (alle anderen 0)

2.2 Arduino-Loop vs. FreeRTOS (Prinzip, nicht Code)

2.2.1 Variante A: Arduino-Loop (ohne eigene Tasks)

`loop()` läuft zyklisch (Polling), z. B. alle 1 ms. Alle Schritte passieren sequentiell:

```
Inputs lesen -> normalisieren -> entprellen -> Event erkennen -> LED schreiben
```

Tabelle 17: Arduino-Loop: Stärken und Risiken

Stärke	Simpel, wenige Nebenläufigkeitsprobleme
Risiko	`loop()` darf nicht blockieren (Regel 3)

2.2.2 Variante B: FreeRTOS (Task-basiert)

- **IO-Task (high prio)** besitzt SPI und macht:
  Inputs lesen → normalisieren → entprellen → `selected_id` bestimmen
- Optional: **App-Task** bekommt Events per Queue (z. B. für Serial/Web)

Tabelle 18: FreeRTOS: Stärken und Risiken

Stärke	Klare Prioritäten, gute Trennung IO ↔ App
Risiko	Ownership/Synchronisation sauber halten (Regel 6)

2.3 One-Hot-Ausgabe (Definition, byte-basiert)

Regel: Erst löschen, dann genau 1 Bit setzen.

```
// Pseudo-Code
clear(led_bytes);
if (selected_id != 0) {
    uint8_t bit = selected_id - 1;
    led_bytes[bit / 8] |= (1u << (bit % 8));
}
```

Passend zum verifizierten Mapping

- `OUT[0]`: LED1..LED8 → Bit0..Bit7
- `OUT[1]`: LED9..LED10 → Bit0..Bit1

2.4 One-Hot-Checkliste (damit es wirklich immer stimmt)

☐ `selected_id` ist **ein** Wert (`0..10`), kein Bitfeld

☐ Active-low wird **einmal** normalisiert (`pressed = !raw_bit`)

☐ Event ist **Press-Edge** (released → pressed), nicht „level“

☐ LED-Ausgabe hat **einen zentralen Pfad**: `clear` → `set one bit`

☐ Reset/Boot setzt deterministischen Zustand (z. B. `selected_id=0`, alle LEDs aus)

☐ Debug/Logs: nur Events loggen (nicht jeden Scan)

3 Embedded-Entwicklung – Engineering-Regeln

3.1 Begriffsrahmen und Zielsetzung

Embedded-Entwicklung unterscheidet sich von Desktop-Software durch drei Eigenschaften: begrenzte Ressourcen, harte Echtzeitanforderungen, physische Schnittstellen. Die folgenden Regeln sind keine abstrakten Best Practices, sondern konkrete Antworten auf konkrete Probleme, die bei der Arbeit mit ESP32-S3, CD4021 und 74HC595 auftreten.

Begriff	Definition
Determinismus	Gleiches Eingangssignal führt immer zum gleichen Verhalten
SSOT	Single Source of Truth – jede Information hat genau eine Quelle
Hot-Path	Code, der in jedem Scan-Zyklus ausgeführt wird
Edge	Zustandsübergang (Flanke), nicht der Zustand selbst
Invariante	Bedingung, die zu jedem Zeitpunkt gelten muss

Tabelle 19: Begriffsrahmen Embedded-Entwicklung

Ziel: Code, der deterministisch läuft, isoliert testbar ist und von 10 auf 100 Kanäle skaliert, ohne dass du die Logik ändern musst.

3.2 Hardware ist die API

Problem: In Desktop-Software definierst du Interfaces mit Methoden und Typen. In Embedded-Systemen ist die „API“ physisch: Pins, Spannungspegel, Timing-Constraints. Diese Schnittstelle ist nicht verhandelbar.

Regel: Behandle Hardware-Konfiguration mit derselben Sorgfalt wie Software-Interfaces – dokumentiert, versioniert, an einer Stelle definiert.

SSOT im Projekt:

Aspekt	SSOT-Datei	Beispielinhalt
Pins	`firmware/include/hw/board_pins.h`	`PIN_SCK = D8`
Counts/Masks	`firmware/include/hw/io_counts.h`	`LED_COUNT = 10`
SPI-Parameter	`firmware/include/hw/spi_params.h`	`CD4021_FREQ = 500000`

Tabelle 20: SSOT-Dateien für Hardware-Konfiguration

Konsequenz: Wenn du einen Pin änderst, änderst du ihn in `board_pins.h` – nirgendwo sonst. Der Compiler erzwingt Konsistenz, weil alle anderen Dateien diese Header einbinden.

Verboten:

```
// Schlecht: Pin hardkodiert in Treiber
void cd4021_init() {
```

```
    pinMode(D1, OUTPUT);  // Wo kommt D1 her? Was ist D1?
}

// Gut: Pin aus SSOT
#include "hw/board_pins.h"
void cd4021_init() {
    pinMode(PIN_BTN_PS, OUTPUT);  // Selbstdokumentierend
}
```

3.3 Determinismus vor Flexibilität

Problem: Embedded-Systeme müssen in definierten Zeitfenstern reagieren. Ein Scan-Zyklus von 10 ms bedeutet: Nach 10 ms muss die nächste Messung starten – nicht „ungefähr 10 ms“, nicht „wenn gerade Zeit ist“.

Regel: Der Worst-Case zählt, nicht der Durchschnitt. Wenn eine Operation manchmal 5 ms und manchmal 50 ms dauert, ist sie im 10 ms-Zyklus unbrauchbar.

Scan-Zyklus-Struktur:

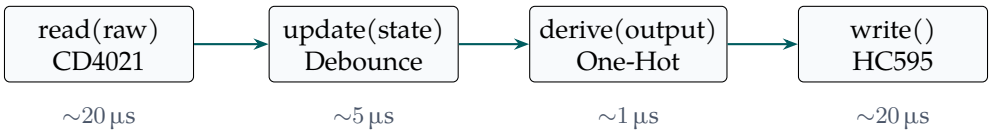


Abbildung 6: Scan-Zyklus-Struktur mit Timing

Timing-Budget (10 ms = 10 000 µs)**:**

Phase	Dauer	Anteil
read()	∼20 µs	0,2 %
update()	∼5 µs	0,05 %
derive()	∼1 µs	0,01 %
write()	∼20 µs	0,2 %
Summe	∼46 µs	∼0,5 %
Reserve	∼9954 µs	99,5 %

Tabelle 21: Timing-Budget pro Scan-Zyklus

Was zerstört Determinismus:

FreeRTOS-Beispiel:

```
// Schlecht: Drift akkumuliert
void io_task(void* param) {
    while (true) {
        scan_cycle();
        vTaskDelay(pdMS_TO_TICKS(10));  // 10 ms NACH Ende des Zyklus
```

Gut	Schlecht	Warum schlecht
`vTaskDelayUntil()`	`vTaskDelay()`	Drift akkumuliert
Logging bei Events	Logging pro Scan	UART blockiert
Feste Zykluszeit	`delay()` im Hot-Path	Timing wird unvorhersagbar

Tabelle 22: Determinismus-Killer vermeiden

```
    }
}

// Gut: Feste Periode
void io_task(void* param) {
    TickType_t lastWake = xTaskGetTickCount();
    while (true) {
        scan_cycle();
        vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(10));  // 10 ms AB Start
    }
}
```

3.4 Edges statt Pegel

Problem: Wenn du auf den Pegel „Button ist gedrückt“ reagierst, feuert die Logik in jedem Scan-Zyklus. Bei 10 ms Intervall sind das 100 Events pro Sekunde, solange der Benutzer die Taste hält.

Regel: UI-Aktionen reagieren auf Flanken (Edges), nicht auf Pegel.

Visualisierung:

```
Pegel:    ****************************  (Taste 500 ms gehalten)
          ^                           ^
        Press                     Release

Edge:     ^                           ^   (zwei Events)
```

Umsetzung im Projekt:

CD4021 (Pegel) → Debounce (stabil) → Edge-Detection → Selection

Code-Beispiel:

```
typedef struct {
    uint8_t stable;      // Letzter stabiler Zustand
    uint8_t previous;    // Fuer Edge-Detection
} ButtonState;

bool detectPressEdge(ButtonState* state, uint8_t current, uint8_t mask) {
    // Debounce hier ausgelassen fuer Klarheit

```

```
    bool wasPressed = (state->previous & mask) == 0;  // Active-low
    bool isPressed  = (current & mask) == 0;

    state->previous = current;

    // Press-Edge: war nicht gedrueckt, ist jetzt gedrueckt
    return (!wasPressed && isPressed);
}
```

Konsequenz

Die Selection-Logik sieht nur „Button 3 wurde gerade gedrückt“, nicht „Button 3 ist seit 500 ms gedrückt“. Das macht das Verhalten vorhersagbar und testbar.

3.5 One-Hot ist Pflicht

Problem: Bei einem Selection Panel mit 100 LEDs wäre es fatal, wenn versehentlich zwei LEDs gleichzeitig leuchten. Das kann passieren, wenn du LEDs an verschiedenen Stellen im Code setzt/löschst.

Regel: One-Hot-Zustände entstehen an genau einer Stelle im Code – nicht verteilt.

Invariante:

- `selectedId == 0` → alle LEDs aus
- `selectedId` ∈ `[1..N]` → genau LED `selectedId` an

Zentrale Implementierung:

```
void setOneHot(uint8_t selectedId, uint8_t* ledState, size_t bytes) {
    // 1. ALLE Bits loeschen (garantiert sauberen Zustand)
    memset(ledState, 0x00, bytes);

    // 2. Genau EIN Bit setzen (wenn ID > 0)
    if (selectedId >= 1 && selectedId <= LED_COUNT) {
        uint8_t byteIdx = (selectedId - 1) / 8;
        uint8_t bitPos  = 7 - ((selectedId - 1) % 8);  // MSB-first
        ledState[byteIdx] |= (1u << bitPos);
    }

    // 3. Ungueltige Bits im letzten Byte maskieren
    ledState[bytes - 1] &= LED_LAST_MASK;
}
```

Warum `memset` zuerst?

Alternative Ansätze wie „altes Bit löschen, neues Bit setzen“ können bei Bugs, Race Conditions oder vergessenen Aufrufen zu Zuständen mit zwei gesetzten Bits führen:

```
// Gefaehrlich: Zwei Operationen, dazwischen kann etwas schiefgehen
ledState[oldByte] &= ~oldMask;  // Was wenn hier ein Interrupt kommt?
ledState[newByte] |= newMask;   // Jetzt sind kurzzeitig zwei Bits gesetzt

// Sicher: Atomare Operation auf dem Zustand
memset(ledState, 0, bytes);     // Alles aus
ledState[byteIdx] |= mask;      // Genau eins an
```

Verboten

```
// Schlecht: LEDs "nebenbei" setzen
void handleSpecialCase() {
    ledState[0] |= 0x80;  // LED 1 zusaetzlich an - bricht One-Hot!
}
```

3.6 Skalierung durch Formeln

Problem: Code mit hardkodierten Werten wie `uint8_t leds[2]` oder `for (int i = 0; i < 10; i++)` muss bei Skalierung manuell angepasst werden. Das ist fehleranfällig.

Regel: Alle Größen, die von der Kanalzahl abhängen, werden aus `*_COUNT` abgeleitet.

SSOT-Definitionen:

```
// io_counts.h
#define BTN_COUNT  10
#define LED_COUNT  10

// Abgeleitete Werte (automatisch korrekt)
#define BTN_BYTES  ((BTN_COUNT + 7) / 8)  // = 2
#define LED_BYTES  ((LED_COUNT + 7) / 8)  // = 2

// Maske fuer ungueltige Bits im letzten Byte
// Bei 10 Kanaelen: 10 % 8 = 2 gueltige Bits -> Maske = 0xC0
#define LED_LAST_MASK  (0xFF << (8 - (LED_COUNT % 8 == 0 ? 8 : LED_COUNT % 8)))
```

Vergleich:

Richtig	Falsch	Warum falsch
`LED_COUNT`	`10`	Bei Änderung vergessen
`LED_BYTES`	`2`	Inkonsistent bei Skalierung
`LED_LAST_MASK`	`0xC0`	Muss neu berechnet werden

Tabelle 23: Richtige vs. falsche Parametrisierung

Skalierungsbeispiel:

Parameter	10 Kanäle	100 Kanäle	Änderung nötig
`LED_COUNT`	10	100	✓ (einmal)
`LED_BYTES`	2	13	Automatisch
`LED_LAST_MASK`	`0xC0`	`0xF0`	Automatisch
Logik-Code	–	–	Keine

Tabelle 24: Skalierung von 10 auf 100 Kanäle

Konsequenz

Skalierung von 10 auf 100 Kanäle erfordert eine Zeile Änderung in `io_counts.h`.

3.7 Schichten einhalten

Problem: Wenn Treiber Geschäftslogik enthalten und Logik Hardware-Zugriffe macht, wird das System untestbar und unwartbar.

Regel: Abhängigkeiten zeigen nach unten. Höhere Schichten nutzen niedrigere, nie umgekehrt.

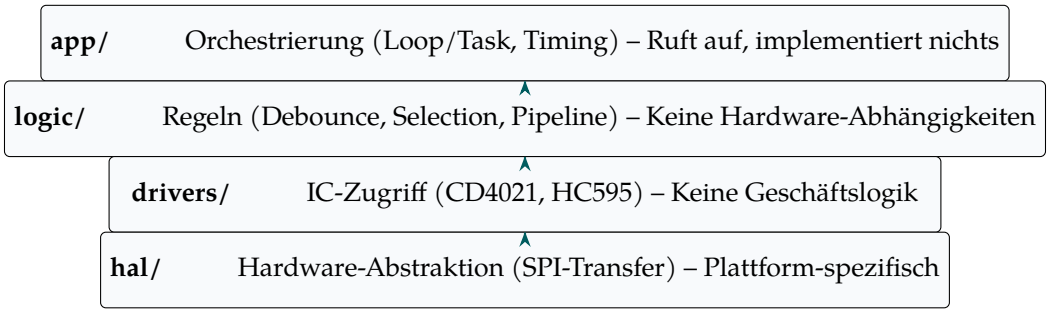


Abbildung 7: Schichtenmodell mit Verantwortlichkeiten

Was darf wo?

Schicht	Darf	Darf nicht
`app/`	Schichten orchestrieren, Timing	Logik implementieren
`logic/`	Zustände verarbeiten, Entscheidungen	Pins kennen, SPI aufrufen
`drivers/`	Bytes lesen/schreiben	Debounce-Entscheidungen
`hal/`	Register beschreiben	IC-Protokolle kennen

Tabelle 25: Zuständigkeiten der Schichten

Warum ist das wichtig?

Die `logic/`-Schicht ist hardware-unabhängig. Du kannst sie auf dem Host-PC testen:

```
1 // test_selection.cpp - laeuft ohne ESP32
2 void test_selection_toggles_on_repress() {
```

```
    SelectionState state = {0};

    // Simuliere: Button 3 gedrueckt
    selection_update(&state, 3, true);
    TEST_ASSERT_EQUAL(3, state.selectedId);

    // Simuliere: Button 3 nochmal gedrueckt -> Toggle aus
    selection_update(&state, 3, true);
    TEST_ASSERT_EQUAL(0, state.selectedId);
}
```

3.8 Testen ist Pflicht

Problem: Embedded-Bugs sind schwer zu debuggen. Du siehst nicht, was auf der Leitung passiert. Ohne Tests merkst du Regressionen erst, wenn das Gerät falsch reagiert.

Regel: Jede Änderung erfordert den passenden Test.

Zweistufige Strategie:

Test-Typ	Befehl	Testet
Unit	`pio test -e seeed_xiao_esp32s3_test -v`	Logik isoliert
E2E	`pio test -e seeed_xiao_esp32s3_e2e -f test_e2e_hw -v`	Hardware-Integration

Tabelle 26: Test-Strategie

Wann welcher Test?

Änderung	Unit-Test	E2E-Test
Debounce-Algorithmus	✓	–
Selection-Policy	✓	–
Pin-Zuordnung	–	✓
SPI-Timing	–	✓
Neuer Kanal-Count	✓	✓

Tabelle 27: Test-Zuordnung nach Änderungstyp

Unit-Test-Beispiel:

```
void test_debounce_filters_noise() {
    DebounceState state = {.stable = 0xFF, .counter = 0};

    // Prellen simulieren: 1 -> 0 -> 1 -> 0 -> 0 -> 0
    TEST_ASSERT_FALSE(debounce_update(&state, 0xFE, 3));  // Zaehler = 1
    TEST_ASSERT_FALSE(debounce_update(&state, 0xFF, 3));  // Reset
    TEST_ASSERT_FALSE(debounce_update(&state, 0xFE, 3));  // Zaehler = 1
    TEST_ASSERT_FALSE(debounce_update(&state, 0xFE, 3));  // Zaehler = 2
```

```
    TEST_ASSERT_TRUE(debounce_update(&state, 0xFE, 3));   // Zaehler = 3 -> Edge!
}
```

3.9 Debugging-Strategie

Regel: Debugge von unten nach oben – Hardware vor Software.

Reihenfolge:

1. **Verkabelung** – Spannung, GND, Signale korrekt?
2. **SPI-Signale** – Clock, MOSI, MISO sichtbar?
3. **Raw-Daten** – Erwartete Bytes vom CD4021?
4. **Logik-Zustand** – Debounce, Selection korrekt?
5. **Output** – LEDs wie erwartet?

Werkzeuge:

Ebene	Werkzeug	Prüfung
Verkabelung	Multimeter	VCC = 3,3 V? GND verbunden?
SPI	Logic Analyzer	Timing, Pegel, Protokoll
Raw-Daten	Serial Monitor	`RAW: 0xFD 0xFF`
Logik	Unit-Tests	Algorithmen isoliert
Output	Augen	LED 5 leuchtet?

Tabelle 28: Debugging-Werkzeuge nach Ebene

Typische Fehlerbilder:

Symptom	Wahrscheinliche Ursache	Prüfung
RAW = 0xFF (immer)	Pullups fehlen oder PS falsch	Verkabelung
RAW = 0x00 (immer)	Kurzschluss nach GND	Verkabelung
Falsche ID erkannt	MSB/LSB vertauscht	Bit-Mapping prüfen
LEDs flackern	Timing zu schnell	SPI-Frequenz senken
Doppelte Events	Debounce zu kurz	Threshold erhöhen

Tabelle 29: Typische Fehlerbilder und Diagnose

3.10 Häufige Fehler

3.11 Checkliste vor Commit

☐ Pins/Counts nur in SSOT-Dateien (`include/hw/*`)

Fehler	Konsequenz	Lösung
`delay()` im Scan	Timing gestört	`millis()`-basiert oder `vTaskDelayUntil()`
Logging pro Scan	Serial überlastet (4 ms pro Zeile)	Nur bei Events loggen
Globale Variablen	Race Conditions bei Interrupts	Zentraler State mit Mutex
Feste Array-Größen	Nicht skalierbar	`*_BYTES` Makros verwenden
LEDs verteilt setzen	One-Hot gebrochen	Zentrale `setOneHot()` Funktion
Pin in Treiber	SSOT verletzt	`board_pins.h` verwenden

Tabelle 30: Häufige Fehler und Lösungen

- ☐ Kein `delay()` im Scan-Pfad
- ☐ One-Hot zentral in einer Funktion erzeugt
- ☐ Letztes Byte mit `*_LAST_MASK` maskiert
- ☐ `logic/` enthält keine Hardware-Includes
- ☐ Unit-Tests grün: `pio test -e seeed_xiao_esp32s3_test -v`
- ☐ Bei HW-Änderung: E2E grün: `pio test -e seeed_xiao_esp32s3_e2e -f test_e2e_hw -v`
- ☐ Dokumentation aktualisiert (bei Änderungen an Wiring, Architektur)

3.12 Weiterführende Dokumentation

`firmware/docs/architecture.md` Schichtenmodell-Details

`firmware/docs/wiring.md` Hardware-Verdrahtung

`firmware/docs/test-plan.md` Testabdeckung

`firmware/docs/debug-playbook.md` Erweiterte Fehlerbehebung

`docs/embedded-grundlagen.md` Konzeptuelle Grundlagen

Teil II
Das Selection-Panel-Projekt

Architektur, Protokolle, Web-Integration

Projektüberblick

Dieser Teil dokumentiert das Selection-Panel-Projekt: Systemarchitektur, Kommunikationsprotokolle und Web-Integration. Die Kapitel bauen aufeinander auf – vom Gesamtbild zu den technischen Details.

4 Selection-Panel-Projekt – Gesamtübersicht

4.1 Begriffsrahmen und Zielsetzung

Projektroot: `/Users/jan/daten/IoT/embedded_projekt`

Zielbild: Ein interaktives Selection Panel mit 10–100 Buttons und LEDs, bei dem zu jedem Zeitpunkt höchstens eine Auswahl aktiv ist (One-Hot-Prinzip). Das System besteht aus drei Ebenen: Firmware (ESP32-S3), Bridge (Raspberry Pi), Dashboard (Browser).

Tabelle 31: Zentrale Begriffe

Begriff	Definition
Selection Panel	Hardware-Interface mit Tastern und LEDs für Kanalauswahl
One-Hot	Invariante: genau 0 oder 1 Auswahl aktiv, niemals mehrere
CD4021	8-Bit Parallel-zu-Seriell Schieberegister (Button-Input)
74HC595	8-Bit Seriell-zu-Parallel Schieberegister (LED-Output)
Bridge	Python-Server, der WebSocket ↔ Serial übersetzt
SSOT	Single Source of Truth – jede Konfiguration existiert an genau einer Stelle

4.2 Systemarchitektur – Drei Ebenen

Die Architektur folgt einer Dreischichten-Struktur: Browser-Dashboard kommuniziert via WebSocket mit der Bridge, die Bridge übersetzt zwischen JSON und binärem Serial-Protokoll zur Firmware.

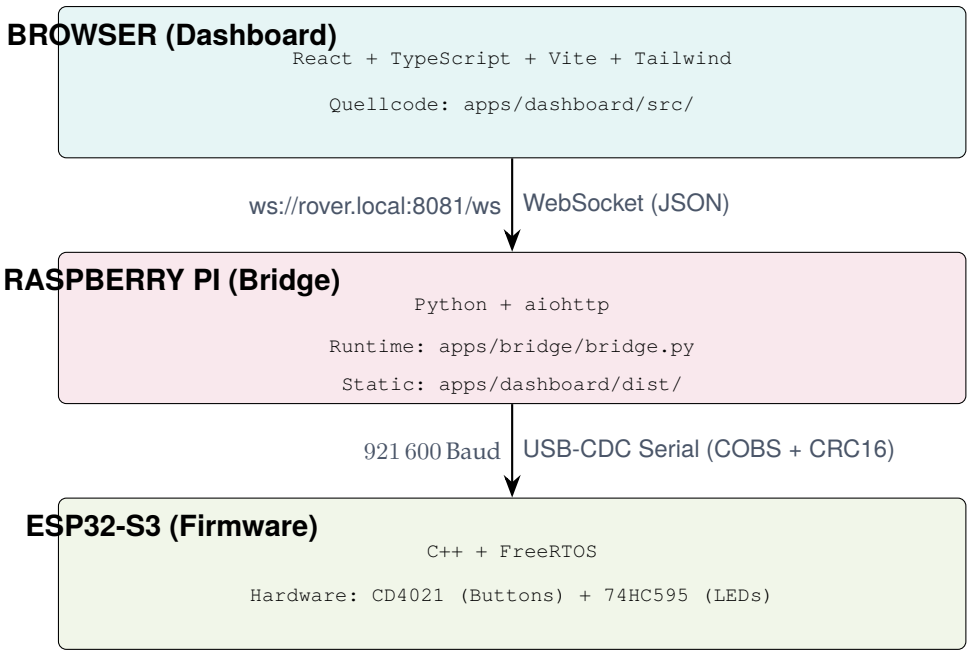


Abbildung 8: Dreischichten-Architektur des Selection Panels

Datenfluss bei Button-Druck

Der Ablauf vom physischen Tastendruck bis zur UI-Aktualisierung durchläuft fünf Stufen:

1. CD4021 liest Button-Matrix (alle 10 ms)
2. Firmware erkennt Press-Edge, aktualisiert `selectedId`
3. Firmware sendet `SELECTION_STATE` via Serial (COBS/CRC)
4. Bridge empfängt, wandelt in JSON: `{"type:ßelection",ßelected:7}`
5. Browser empfängt via WebSocket, React aktualisiert UI

4.3 Repository-Struktur

Das Repository gliedert sich in drei Hauptbereiche: Firmware, Apps (Bridge und Dashboard) sowie Hardware-Designs und Dokumentation.

Listing 1: Repository-Struktur

```
embedded_projekt/
|-- apps/
|   |-- bridge/                    # Python Bridge (Pi Runtime)
|   |   |-- bridge.py              # HTTP + WebSocket + Serial
|   |   |-- docs/
|   |   |   |-- PROTOCOL.md        # Serial/WS Protokoll (SSOT)
|   |   |   |-- WORKFLOW.md        # Build/Deploy/Run
|   |   |   `-- SMOKE_TEST.md      # E2E-Tests
|   |   `-- requirements.txt
|   |
|   `-- dashboard/                 # React Frontend
|       |-- src/
|       |   |-- pages/             # DashboardPage, SettingsPage
|       |   |-- components/        # UI-Komponenten
|       |   `-- lib/               # protocol.ts, ws.ts
|       |-- dist/                  # Build-Output (wird von Bridge served)
|       |-- docs/
|       |   |-- DEVELOPER.md       # Frontend-Entwicklung
|       |   `-- USER_GUIDE.md      # Benutzerhandbuch
|       |-- vite.config.ts
|       `-- package.json
|
|-- firmware/                      # ESP32-S3 Firmware (C++)
|   |-- include/hw/                # SSOT: Pins, Counts, SPI-Parameter
|   |-- src/
|   |   |-- hal/                   # SPI-Bus Zugriff
|   |   |-- drivers/               # CD4021, HC595
|   |   |-- logic/                 # Debounce, Selection, Pipeline
|   |   |-- app/                   # Loop/Task Orchestrierung
|   |   `-- protocol/              # COBS/CRC Frames
|   |-- test/                      # Unit + E2E Tests
```

```
|   |-- docs/                    # Firmware-Dokumentation
|   `-- platformio.ini
|
|-- hardware/
|   `-- kicad/                   # Schaltplaene, PCB
|
|-- doc/
|   |-- md/                      # Markdown-Quellen (SSOT)
|   `-- latex/                   # LaTeX/PDF-Output
|
|-- assets/
|   |-- pdf/                     # Generierte PDFs
|   `-- video/                   # Medien
|
`-- scripts/                      # Build-/Deploy-Helfer
```

4.4 SSOT – Single Source of Truth

Regel: Jede Konfiguration existiert an genau einer Stelle. Änderungen propagieren automatisch.

Tabelle 32: SSOT-Dateien nach Aspekt

Aspekt	SSOT-Datei
Pins	`firmware/include/hw/board_pins.h`
Counts/Masks	`firmware/include/hw/io_counts.h`
SPI-Parameter	`firmware/include/hw/spi_params.h`
Serial-Protokoll	`apps/bridge/docs/PROTOCOL.md`
WebSocket-Protokoll	`apps/bridge/docs/PROTOCOL.md`

Tabelle 33: Generierte Artefakte

Artefakt	Quelle	Output
Frontend	`apps/dashboard/src/`	`apps/dashboard/dist/`
PDFs	`doc/latex/`	`assets/pdf/`

4.5 Firmware – Build, Flash, Test

Flashen

Listing 2: Firmware flashen

```
cd /Users/jan/daten/IoT/embedded_projekt

# FreeRTOS-Variante (empfohlen)
```

```
pio run -d firmware -e seeed_xiao_esp32s3_freertos -t upload

# Arduino-Loop-Variante
pio run -d firmware -e seeed_xiao_esp32s3_loop -t upload
```

Monitor

```
pio device monitor -d firmware -e seeed_xiao_esp32s3_freertos
```

Tests

Listing 3: Unit- und E2E-Tests

```
# Unit-Tests (Logik: Debounce, Selection, Pipeline)
pio test -d firmware -e seeed_xiao_esp32s3_test -v

# E2E-Hardwaretest (SPI, Pinning, Timing)
pio test -d firmware -e seeed_xiao_esp32s3_e2e -f test_e2e_hw -v
```

Tabelle 34: Build-Varianten

Environment	Beschreibung
`seeed_xiao_esp32s3_freertos`	FreeRTOS Task (`APP_USE_FREERTOS=1`)
`seeed_xiao_esp32s3_loop`	Arduino Loop (`APP_USE_FREERTOS=0`)
`seeed_xiao_esp32s3_test`	Unit-Tests (nur `logic/`)
`seeed_xiao_esp32s3_e2e`	E2E-Hardware-Test

4.6 Dashboard – Entwicklung und Build

DEV-Server (Mac)

```
cd /Users/jan/daten/IoT/embedded_projekt/apps/dashboard
npm ci
npm run dev
# -> http://localhost:5173/
# -> WebSocket via Proxy zu rover.local:8081
```

Production-Build

```
npm run lint
npm run typecheck
npm run build
# -> Output: dist/
```

Tabelle 35: npm Scripts

Script	Zweck
`npm run dev`	DEV-Server mit HMR
`npm run lint`	ESLint-Prüfung
`npm run typecheck`	TypeScript-Prüfung
`npm run build`	Production-Build nach `dist/`
`npm run format`	Prettier-Formatierung

4.7 Bridge – Raspberry Pi

Manueller Start

Listing 4: Bridge manuell starten

```
ssh pi@rover
source ~/venvs/selection-bridge/bin/activate

python /home/pi/embedded_projekt/apps/bridge/bridge.py \
  --port auto \
  --vid 0x303A --pid 0x1001 \
  --baud 921600 \
  --http 8081 \
  --web-root /home/pi/embedded_projekt/apps/dashboard/dist \
  --wait-port --wait-timeout 0 --wait-poll 0.5 \
  --reconnect-delay 2 \
  --ack-timeout 0.6 --retries 4
```

systemd-Service

Listing 5: systemd-Befehle

```
# Status pruefen
sudo systemctl status embedded-bridge.service --no-pager -l

# Neu starten
sudo systemctl restart embedded-bridge.service

# Logs verfolgen
sudo journalctl -u embedded-bridge.service -f
```

Tabelle 36: URLs im PROD-Modus

Zweck	URL
Dashboard	http://rover.local:8081/
WebSocket	ws://rover.local:8081/ws

4.8 Deployment-Workflow

Mac → Pi synchronisieren

Listing 6: rsync-Synchronisation

```
# Zielpfad auf dem Pi anlegen (einmalig)
ssh pi@rover 'mkdir -p /home/pi/embedded_projekt/apps/bridge \
                       /home/pi/embedded_projekt/apps/dashboard'

# Sync (ohne venv, node_modules, dist)
rsync -avh --delete \
  --exclude '.git/' \
  --exclude '.venv/' \
  --exclude '__pycache__/' \
  --exclude '*.pyc' \
  --exclude '.DS_Store' \
  --exclude 'node_modules/' \
  --exclude 'apps/dashboard/dist/' \
  ./ \
  pi@rover:/home/pi/embedded_projekt/
```

Dashboard dist sync

Listing 7: Dashboard-Build und Sync

```
# Lokal bauen
cd apps/dashboard
npm ci
npm run build
cd -

# Dashboard dist sync (prod)
rsync -avh --delete \
  --exclude '.DS_Store' \
  ./apps/dashboard/dist/ \
  pi@rover:/home/pi/embedded_projekt/apps/dashboard/dist/
```

Service neu starten

```
ssh pi@rover "sudo systemctl daemon-reload && \
              sudo systemctl restart embedded-bridge.service"
```

4.9 Dokumentationsstruktur

Firmware-Dokumentation

Die Firmware-Dokumentation liegt unter `firmware/docs/`:

Tabelle 37: Firmware-Dokumentation

Datei	Inhalt
`overview.md`	Projektübersicht, Bit-Mapping (MSB-first)
`wiring.md`	Hardware-Verdrahtung (Pins, ICs)
`architecture.md`	Schichtenmodell (HAL → Driver → Logic → App)

Bridge-Dokumentation

Die Bridge-Dokumentation liegt unter `apps/bridge/docs/`:

Tabelle 38: Bridge-Dokumentation

Datei	Inhalt
`PROTOCOL.md`	Serial + WebSocket Protokoll (SSOT)
`WORKFLOW.md`	Build/Deploy/Run
`SMOKE_TEST.md`	E2E-Tests

Dashboard-Dokumentation

Die Dashboard-Dokumentation liegt unter `apps/dashboard/docs/`:

Tabelle 39: Dashboard-Dokumentation

Datei	Inhalt
`DEVELOPER.md`	Frontend-Entwicklung
`USER_GUIDE.md`	Benutzerhandbuch

4.10 Typische Arbeitsabläufe

Firmware-Logik ändern

1. Code ändern in `firmware/src/logic/`
2. Unit-Tests: `pio test -d firmware -e seeed_xiao_esp32s3_test -v`
3. Bei Erfolg: Flashen mit `pio run ... -t upload`
4. Monitor: `pio device monitor -d firmware`

Dashboard-UI ändern

1. DEV-Server starten: `cd apps/dashboard && npm run dev`
2. Ändern – Browser aktualisiert automatisch (HMR)
3. Lint + Build: `npm run lint && npm run build`
4. Deploy (siehe Abschnitt 4.8)

Hardware-Verdrahtung ändern

Bei Änderungen an der Hardware-Verdrahtung:

1. SSOT aktualisieren: `firmware/include/hw/board_pins.h`
2. Dokumentation aktualisieren: `firmware/docs/wiring.md`
3. E2E-Test: `pio test ... -e seeed_xiao_esp32s3_e2e -f test_e2e_hw -v`

4.11 Roadmap

Tabelle 40: Projekt-Roadmap

Milestone	Status	Beschreibung
10× Buttons/LEDs	✓	Grundfunktion verifiziert
Serial-Protokoll	✓	COBS/CRC, ACK/Retry
React-Dashboard	✓	DEV/PROD-Split
100× Skalierung	–	Counts/Masks + Kaskadierung + Timing
Audio-Integration	–	Multimedia bei Selection-Change

4.12 Siehe auch

Weiterführende Dokumentation:

- `firmware/docs/overview.md` – Firmware-Einstieg
- `apps/dashboard/docs/DEVELOPER.md` – Frontend-Einstieg
- `apps/bridge/docs/WORKFLOW.md` – Pi-Deployment
- `apps/bridge/docs/PROTOCOL.md` – Protokoll-Spezifikation

5 Serial-Protokoll – Robuste Frames mit COBS + CRC

5.1 Begriffsrahmen und Zielsetzung

Der ESP32-S3 kommuniziert mit dem Raspberry Pi über USB-CDC (virtueller Serial-Port). Die Herausforderung: Serielle Kommunikation ist ein kontinuierlicher Byte-Strom ohne inhärente Nachrichtengrenzen.

Tabelle 41: Zentrale Begriffe des Serial-Protokolls

Begriff	Definition
USB-CDC	USB Communication Device Class – emuliert einen seriellen Port über USB
Frame	Ein abgegrenztes Datenpaket mit definiertem Anfang und Ende
COBS	Consistent Overhead Byte Stuffing – Framing-Verfahren mit `0x00` als Delimiter
CRC-16	Cyclic Redundancy Check – Prüfsumme zur Fehlererkennung
Payload	Nutzdaten innerhalb eines Frames
ACK	Acknowledgment – Empfangsbestätigung

Ziel

Zuverlässige, bidirektionale Kommunikation mit Fehlererkennung und Retry-Mechanik.

5.2 Das Grundproblem – Byte-Strom vs. Nachrichten

Wenn du Bytes über eine serielle Leitung schickst, sieht der Empfänger nur einen kontinuierlichen Strom:

```
Sender:   [Nachricht A: 5 Bytes][Nachricht B: 3 Bytes][Nachricht C: 7 Bytes]
Leitung:  0x12 0x34 0x56 0x78 0x9A 0xBC 0xDE 0xF0 0x11 0x22 0x33 0x44 0x55 0x66 0x77
Empfänger: "Wo fängt Nachricht B an?"
```

Ohne Framing weiß der Empfänger nicht, wo eine Nachricht endet und die nächste beginnt. Drei klassische Lösungsansätze existieren:

Tabelle 42: Vergleich der Framing-Ansätze

Ansatz	Prinzip	Nachteil
Feste Länge	Jede Nachricht hat exakt N Bytes	Unflexibel, verschwendet Bandbreite
Längenfeld	Erstes Byte gibt Länge an	Synchronisationsverlust bei Byte-Verlust
Delimiter	Spezielles Zeichen markiert Ende	Was, wenn das Zeichen in den Daten vorkommt?

COBS kombiniert den Delimiter-Ansatz mit Byte-Stuffing – das Trennzeichen `0x00` kann garantiert nicht in den Daten vorkommen.

5.3 COBS – Consistent Overhead Byte Stuffing

Wir transformieren die Daten so, dass `0x00` niemals im Inhalt auftritt. Dann können wir `0x00` als eindeutigen Delimiter verwenden.

Wie COBS funktioniert

COBS ersetzt jede `0x00` in den Originaldaten durch einen „Zeiger“ auf die nächste Null (oder das Paketende):

```
Original:     [0x11] [0x22] [0x00] [0x33] [0x00] [0x44]
                             ^             ^
                          Nullen müssen weg

COBS-kodiert: [0x03] [0x11] [0x22] [0x02] [0x33] [0x02] [0x44]
               |                    |             |
               +-- "nach 3 Bytes    +-- "nach 2   +-- "nach 2 Bytes
                   kommt Ersatz"        Bytes"        kommt Ende"
```

Der Empfänger liest den ersten Byte-Wert (hier `0x03`), kopiert die folgenden 2 Bytes, fügt eine `0x00` ein, liest den nächsten Zeiger, und so weiter.

Wire-Frame-Aufbau

```
+-------------------------------------------+-----------+
|  COBS-kodierte Daten (keine 0x00 enthalten) |   0x00    |
|                                           | Delimiter |
+-------------------------------------------+-----------+
```

COBS fügt im Worst Case 1 B pro 254 B Originaldaten hinzu. Bei typischen Paketgrößen unter 100 B ist der Overhead vernachlässigbar (1–2 Bytes).

Konsequenz für den Empfänger

Du sammelst Bytes, bis du `0x00` siehst. Dann dekodierst du alles davor mit COBS und hast das Original-Paket.

5.4 CRC-16 – Integritätsprüfung

Auch mit korrektem Framing können Bytes auf der Leitung verfälscht werden – durch EMI, Timing-Probleme oder Buffer-Überläufe. Der Sender berechnet eine Prüfsumme über die Daten und hängt sie an. Der Empfänger berechnet dieselbe Prüfsumme und vergleicht.

CRC-16/CCITT-FALSE Parameter

Tabelle 43: CRC-16/CCITT-FALSE Konfiguration

Parameter	Wert	Bedeutung
Polynom	`0x1021`	Mathematische Basis der Berechnung
Init	`0xFFFF`	Startwert
XorOut	`0x0000`	Finaler XOR (hier: keiner)
RefIn/RefOut	`false`	Keine Bit-Spiegelung

Dieses Polynom erkennt alle Einzelbitfehler, alle Doppelbitfehler, und die meisten Burst-Fehler bis 16 bit Länge. Die Erkennungswahrscheinlichkeit für zufällige Fehler liegt bei $1 - 2^{-16} \approx 99{,}998\,\%$.

Berechnung (Python-Referenz)

Listing 8: CRC-16/CCITT-FALSE Implementierung

```python
def crc16_ccitt_false(data: bytes) -> int:
    crc = 0xFFFF
    for b in data:
        crc ^= (b << 8) & 0xFFFF
        for _ in range(8):
            if crc & 0x8000:
                crc = ((crc << 1) ^ 0x1021) & 0xFFFF
            else:
                crc = (crc << 1) & 0xFFFF
    return crc
```

Wenn CRC nicht stimmt, verwirft der Empfänger das Paket. Der Sender muss es erneut senden (siehe Abschnitt 5.8).

5.5 Packet-Layout – Der Header

Bevor COBS angewendet wird, bauen wir das „Raw-Packet“ mit einem strukturierten Header:

Tabelle 44: Packet-Header-Struktur

Offset	Feld	Größe	Beschreibung
0	magic0	1	`'S'` (`0x53`)
1	magic1	1	`'P'` (`0x50`)
2	ver_major	1	1
3	ver_minor	1	0
4	type	1	Message-Typ
5	flags	1	Bitflags
6–7	seq	2	Sequenznummer (Little-Endian)
8–9	len	2	Payload-Länge (Little-Endian)
10..	payload	len	Nutzdaten
10+len	crc16	2	CRC über Bytes 0..(10+len-1)

Warum diese Felder?

Magic Bytes (`SP`): Wenn wir mitten in einem Datenstrom einsteigen oder ein Frame korrupt ist, erkennen wir sofort, dass etwas nicht stimmt. Die Wahrscheinlichkeit, dass zufällige Daten mit `0x53 0x50` beginnen, liegt bei $1/65536 \approx 0{,}0015\,\%$.

Version: Ermöglicht Protokoll-Evolution. Regel: Major-Mismatch führt zum Verwerfen des Frames. Minor-Mismatch wird akzeptiert (abwärtskompatibel).

Sequenznummer: Identifiziert Pakete für ACK-Zuordnung. Bereich 1–65535, Wert 0 wird übersprungen.

Flags

Tabelle 45: Flag-Bits im Header

Bit	Name	Bedeutung
0	`ACK_REQ`	Sender erwartet Bestätigung
1	`IS_ACK`	Dieses Paket ist eine Bestätigung
2	`IS_ERR`	Dieses Paket meldet einen Fehler

5.6 Sende- und Empfangsprozess

Senden (4 Schritte)

1. **Header + Payload zusammenbauen:** Header (10 B) gefolgt von Payload.
2. **CRC-16 berechnen und anhängen:** CRC über gesamten Inhalt, dann 2 Bytes anfügen.

3. **COBS-encode:** Eliminiert alle `0x00` aus dem Datenstrom.
4. **Delimiter anhängen und senden:** `0x00` markiert das Frame-Ende.

Empfangen (4 Schritte)

1. Bytes sammeln bis `0x00`.
2. COBS-decode.
3. CRC prüfen (letzte 2 Bytes vs. Berechnung).
4. Header parsen, Payload extrahieren.

5.7 Message-Typen

ESP32 → Pi (Events)

Tabelle 46: Event-Typen vom ESP32 zum Pi

Type	Hex	Zweck
HELLO	`0x01`	Fähigkeiten nach Reset melden
BTN_EVENT	`0x10`	Button gedrückt/losgelassen
SELECTION_STATE	`0x12`	Aktuell gewählter Kanal
SNAPSHOT	`0x11`	Kompletter Button/LED-Zustand

Pi → ESP32 (Commands)

Tabelle 47: Command-Typen vom Pi zum ESP32

Type	Hex	Zweck
HELLO_ACK	`0x02`	HELLO bestätigen
CMD_SET_SELECTION	`0x20`	Kanal vom Dashboard setzen
CMD_REQ_SNAPSHOT	`0x23`	Debug-Snapshot anfordern

Bidirektional (Control)

Tabelle 48: Bidirektionale Control-Typen

Type	Hex	Zweck
ACK	`0x30`	Empfangsbestätigung
ERR	`0x50`	Fehler melden
PING	`0x03`	Keepalive-Anfrage
PONG	`0x04`	Keepalive-Antwort

SELECTION_STATE – Das zentrale Event

Dieses Paket synchronisiert den Zustand zwischen ESP32, Pi und Browser:

Tabelle 49: SELECTION_STATE Payload-Struktur

Offset	Feld	Typ	Beschreibung
0–1	selected_id	u16	0 = keiner, 1..100 = Kanal
2	source	u8	Wer hat ausgelöst?
3	reserved	u8	0
4–7	t_ms	u32	`millis()` seit Boot

Tabelle 50: Source-Werte für SELECTION_STATE

Wert	Name	Bedeutung
0	`local_btn`	Physischer Button am Panel
1	`host_cmd`	Dashboard-Klick (via Pi)
2	`reset`	Nach ESP32-Neustart
3	`recover`	Wiederherstellung nach Verbindungsabbruch

5.8 ACK/Retry-Mechanik

Bei Commands wie `CMD_SET_SELECTION` muss der Pi wissen, ob der ESP32 das Kommando erhalten und verarbeitet hat. Die Lösung: Request-Response mit Timeout und Retry.

Ablauf

```
Pi                                ESP32
 |                                  |
 |--- CMD_SET_SELECTION ------------>|  seq=42, ACK_REQ=1
```

```
|          (Timer startet)           |
|                                    |  (verarbeitet...)
|<---------- ACK --------------------|  seq=42, IS_ACK=1
|          (Timer stoppt)            |
|                                    |
|<-- SELECTION_STATE ----------------|  selected_id=7, source=host_cmd
```

Fehlerfall

```
Pi                                 ESP32
 |                                   |
 |--- CMD_SET_SELECTION ------------>|  seq=43, ACK_REQ=1
 |          (Timer: 600ms)           |
 |               ...                 |  (keine Antwort)
 |          (Timeout!)               |
 |--- CMD_SET_SELECTION ------------>|  seq=43, Retry #1
 |               ...                 |
 |          (Timeout!)               |
 |--- CMD_SET_SELECTION ------------>|  seq=43, Retry #2
 |               ...                 |
 |<---------- ACK --------------------|  (endlich!)
```

Parameter (bridge.py)

Tabelle 51: ACK/Retry-Parameter

Parameter	Default	Bedeutung
`-ack-timeout`	0,6 s	Wartezeit auf ACK
`-retries`	4	Maximale Wiederholungen

Fehlerbehandlung

Nach 5 fehlgeschlagenen Versuchen (1 + 4 Retries) meldet die Bridge einen Fehler an den Browser (`error code=21`).

5.9 Datenfluss – Vom Button zum Browser

Wenn du Button 7 am Panel drückst, passiert folgendes:

1. **ESP32:** CD4021BE liest Button-Matrix (alle 10 ms). Debounce erkennt stabile Flanke nach 50 ms.
2. **ESP32:** Baut `SELECTION_STATE`-Paket. Header: `magic=SP`, `type=0x12`, `seq=1234`, `len=8`. Payload: `selected_id=7`, `source=0`, `t_ms=45678`. CRC16 anhängen, COBS-encode, `0x00` Delimiter.
3. **Pi (bridge.py):** Empfängt Frame über USB-CDC Serial. COBS-decode, CRC prüfen, Header parsen. Erkennt `SELECTION_STATE`.

4. **Pi:** Wandelt in JSON für WebSocket:

```
{"type":"selection","selected":7,"source":"local_btn","tMs":45678}
```

5. **Browser (React):** `useBridgeWs` empfängt JSON. State-Update führt dazu, dass LED 7 im Dashboard leuchtet.

Latenz

Vom Button-Druck bis zur Browser-Anzeige vergehen typisch 20 ms bis 50 ms (abhängig von Debounce-Zeit und Netzwerk).

5.10 Größenrahmen und Limits

Tabelle 52: Protokoll-Konstanten und Limits

Konstante	Wert	Bedeutung
`HEADER_LEN`	10 B	Feste Header-Größe
`CRC_LEN`	2 B	CRC-16
`MAX_PAYLOAD`	256 B	Firmware-Limit
`MAX_PACKET`	268 B	Header + Payload + CRC
`MAX_WIRE_FRAME`	∼270 B	Nach COBS + Delimiter

Ein `SELECTION_STATE`-Paket ist 10 B (Header) + 8 B (Payload) + 2 B (CRC) = 20 B vor COBS, etwa 21–22 Bytes on-wire.

5.11 Fehlerbehandlung

Tabelle 53: Fehlerarten und Reaktionen

Fehlerart	Erkennung	Reaktion
CRC-Mismatch	CRC-Prüfung	Frame verwerfen
Magic falsch	Header-Prüfung	Frame verwerfen, Resync
Länge inkonsistent	`len` vs. tatsächliche Größe	Frame verwerfen
ACK-Timeout	Timer	Retry, dann Error
Unbekannter Type	Dispatch	Ignorieren (forward-compatible)

Resynchronisation

Da `0x00` garantiert nur als Delimiter vorkommt, findet der Empfänger nach einem korrupten Frame automatisch den nächsten gültigen Frame-Start.

5.12 Siehe auch

- `apps/bridge/docs/PROTOCOL.md` – SSOT für Wire-Format und WebSocket-Events
- `apps/bridge/bridge.py` – Python-Implementation
- `firmware/src/protocol/` – ESP32-Implementation
- `apps/dashboard/src/lib/` – TypeScript-Typen für WebSocket-Events

6 React-Stack Grundlagen

6.1 Begriffsrahmen und Zielsetzung

Du entwickelst das Selection-Panel-Frontend (Browser-UI), das über WebSocket mit dem Python-Backend kommuniziert. Der Stack ersetzt manuelles HTML/CSS/JS durch ein wartbares, typsicheres System.

Tabelle 54: Zentrale Begriffe des React-Stacks

Begriff	Definition
React	JavaScript-Bibliothek für komponentenbasierte UIs
TypeScript	JavaScript-Erweiterung mit statischer Typisierung
Vite	Build-Tool und Entwicklungsserver (sprich: „viit“, französisch für „schnell“)
ESLint	Statische Code-Analyse (findet Fehler vor Laufzeit)
Prettier	Code-Formatter (einheitlicher Stil)
React Router	Client-seitige Navigation ohne Seitenneuladen
Tailwind CSS	Utility-First CSS-Framework

6.2 React – Komponenten statt DOM-Manipulation

Bei klassischem JavaScript manipulierst du das DOM direkt:

Listing 9: Klassische DOM-Manipulation

```
// Klassisch: imperativ, fehleranfaellig bei Zustandsaenderungen
document.getElementById('led-5').classList.add('active');
document.getElementById('status').textContent = 'LED 5 an';
```

Bei 100 LEDs und komplexen Zuständen wird dieser Ansatz unwartbar. Wenn sich ein Wert ändert, musst du manuell alle betroffenen DOM-Elemente aktualisieren.

React kehrt das Prinzip um – du beschreibst den *gewünschten Zustand*, React berechnet die nötigen DOM-Änderungen:

Listing 10: React-Komponente (deklarativ)

```
1 // React: deklarativ, Zustand -> UI-Ableitung
2 function LedButton({ id, isActive, onPress }: LedProps) {
3   return (
4     <button
5       className={isActive ? 'led active' : 'led'}
6       onClick={() => onPress(id)}
7     >
8       LED {id}
9     </button>
```

```
  );
}
```

Kernkonzept – Komponenten

Eine Komponente ist eine Funktion, die Props (Eingabeparameter) erhält und JSX (HTML-ähnliche Syntax) zurückgibt. Komponenten sind wiederverwendbar und isoliert.

Kernkonzept – State und Hooks

Lokaler Zustand wird mit `useState` verwaltet:

Listing 11: State-Management mit useState

```
function LedPanel() {
  const [activeIds, setActiveIds] = useState<Set<number>>(new Set());

  const handlePress = (id: number) => {
    setActiveIds(prev => {
      const next = new Set(prev);
      next.has(id) ? next.delete(id) : next.add(id);
      return next;
    });
  };

  return (
    <div className="grid">
      {Array.from({ length: 100 }, (_, i) => (
        <LedButton
          key={i}
          id={i}
          isActive={activeIds.has(i)}
          onPress={handlePress}
        />
      ))}
    </div>
  );
}
```

Konsequenz für Selection Panel

Wenn der ESP32 über WebSocket meldet „LED 42 aktiv“, aktualisierst du nur den State. React rendert automatisch nur die betroffene Komponente neu – nicht die anderen 99 LEDs.

6.3 TypeScript – Fehler vor Laufzeit finden

JavaScript-Fehler treten erst zur Laufzeit auf:

Listing 12: JavaScript ohne Typisierung

```
// JavaScript: Fehler erst im Browser sichtbar
function setLedState(id, state) {
  websocket.send({ led: id, state: state }); // Typo? Falscher Typ?
}
setLedState("5", "an"); // Kompiliert, crasht zur Laufzeit
```

TypeScript prüft Typen beim Kompilieren:

Listing 13: TypeScript mit Typisierung

```
// TypeScript: Fehler im Editor, bevor du speicherst
interface LedCommand {
  led: number;
  state: boolean;
}

function setLedState(id: number, state: boolean): void {
  const cmd: LedCommand = { led: id, state };
  websocket.send(JSON.stringify(cmd));
}

setLedState("5", "an"); // Fehler: string ist nicht number
setLedState(5, true);   // korrekt
```

Wichtige Typen für dein Projekt

Listing 14: WebSocket-Nachrichten typisieren

```
// WebSocket-Nachrichten typisieren
type WsMessage =
  | { type: 'button_press'; id: number }
  | { type: 'led_update'; id: number; state: boolean }
  | { type: 'audio_start'; file: string };

// Props fuer Komponenten
interface LedGridProps {
  leds: Map<number, boolean>;
  onLedClick: (id: number) => void;
}
```

Konsequenz

Der Compiler fängt Tippfehler, falsche Argumentreihenfolgen und fehlende Eigenschaften ab. Bei 100 Buttons mit verschiedenen Zuständen reduziert das die Debugging-Zeit erheblich.

6.4 Vite – Build-Tool und Entwicklungsserver

Browser verstehen weder TypeScript noch JSX direkt. Der Code muss transformiert werden:

```
Quellcode (.tsx)  ->  [Transformation]  ->  Browser-Code (.js)
```

Ältere Tools wie Webpack benötigen dafür mehrere Sekunden bis Minuten. Vite nutzt ES-Module (native Browser-Imports) und transformiert nur geänderte Dateien:

Listing 15: Vite-Projekt erstellen

```
npm create vite@latest pi-react -- --template react-ts
cd pi-react
npm install
npm run dev   # Startet localhost:5173, Hot Module Replacement
```

Zwei Modi

Tabelle 55: Vite Build-Modi

Modus	Befehl	Ergebnis
Entwicklung	`npm run dev`	Lokaler Server mit Live-Reload, Source Maps
Produktion	`npm run build`	Optimierte statische Dateien in `dist/`

Konfiguration (vite.config.ts)

Listing 16: Vite-Konfiguration mit WebSocket-Proxy

```
1  import { defineConfig } from 'vite';
2  import react from '@vitejs/plugin-react';
3
4  export default defineConfig({
5    plugins: [react()],
6    server: {
7      proxy: {
8        '/ws': {
9          target: 'ws://localhost:8081',
10         ws: true
11       }
12     }
13   },
14   build: {
15     outDir: 'dist'  // Standard-Output
16   }
17 });
```

Konsequenz

Änderungen im Code erscheinen in unter 100 ms im Browser. Nach `npm run build` liegen statische Dateien in `dist/` bereit, die dein Python-Backend ausliefert.

6.5 ESLint + Prettier – Code-Qualität automatisieren

Inkonsistenter Code (Tabs vs. Spaces, fehlende Semikolons) erschwert Reviews. Potenzielle Bugs bleiben unentdeckt.

ESLint (Analyse)

ESLint findet problematische Muster:

```
npm install -D eslint @typescript-eslint/parser @typescript-eslint/eslint-plugin
```

Listing 17: ESLint-Konfiguration

```
// eslint.config.js
export default [
  {
    files: ['**/*.{ts,tsx}'],
    rules: {
      'no-unused-vars': 'error',
      '@typescript-eslint/no-explicit-any': 'warn',
      'react-hooks/exhaustive-deps': 'warn'
    }
  }
];
```

Prettier (Formatierung)

Prettier erzwingt einheitlichen Stil:

```
npm install -D prettier eslint-config-prettier
```

Listing 18: Prettier-Konfiguration

```
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5"
}
```

Integration in VS Code

Listing 19: VS Code Settings

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
```

```
  }
}
```

Konsequenz

Beim Speichern formatiert Prettier automatisch, ESLint markiert Probleme. Das eliminiert Style-Diskussionen und fängt häufige Fehler früh ab.

6.6 React Router – Client-seitige Navigation

Bei klassischen Webseiten lädt jeder Seitenwechsel die komplette Seite neu. Der WebSocket würde dabei getrennt.

React Router wechselt Komponenten, ohne die Seite neu zu laden:

```
npm install react-router-dom
```

Listing 20: React Router Setup

```
// App.tsx
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Dashboard</Link>
        <Link to="/config">Konfiguration</Link>
        <Link to="/debug">Debug</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Dashboard />} />
        <Route path="/config" element={<ConfigPanel />} />
        <Route path="/debug" element={<DebugView />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Wichtig für Selection Panel

Der WebSocket-Hook lebt oberhalb des Routers, sodass die Verbindung bei Navigation bestehen bleibt:

Listing 21: WebSocket-Context oberhalb des Routers

```
function App() {
  const ws = useWebSocket('ws://localhost:8081/ws');
```

```
3
4   return (
5     <WebSocketContext.Provider value={ws}>
6       <BrowserRouter>
7         {/* Routes hier */}
8       </BrowserRouter>
9     </WebSocketContext.Provider>
10  );
11 }
```

Du kannst zwischen Dashboard, Konfiguration und Debug-Ansicht wechseln, ohne die ESP32-Verbindung zu verlieren.

6.7 Tailwind CSS – Utility-First Styling

Traditionelles CSS erfordert separate Dateien und erfundene Klassennamen:

```
/* styles.css - waechst unkontrolliert */
.led-button { ... }
.led-button-active { ... }
.led-button-disabled { ... }
```

Tailwind bietet vordefinierte Utility-Klassen direkt im Markup:

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Listing 22: Tailwind-Konfiguration

```
1  // tailwind.config.js
2  export default {
3    content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
4    theme: {
5      extend: {
6        colors: {
7          'led-active': '#22c55e',
8          'led-inactive': '#374151'
9        }
10     }
11   }
12 };
```

Anwendung

Listing 23: Tailwind-Klassen in React

```
1  // Statt CSS-Klassen zu erfinden: direkte Utilities
2  function LedButton({ isActive }: { isActive: boolean }) {
3    return (
```

```
    <button className={`
      w-12 h-12 rounded-full
      transition-colors duration-150
      ${isActive
        ? 'bg-led-active shadow-lg shadow-green-500/50'
        : 'bg-led-inactive hover:bg-gray-600'}
    `}>
    </button>
  );
}
```

Grid für 100 LEDs

```
<div className="grid grid-cols-10 gap-2 p-4">
  {leds.map((led, i) => <LedButton key={i} isActive={led} />)}
</div>
```

Konsequenz

Kein Wechsel zwischen Dateien, keine CSS-Spezifitätskonflikte. Die Bundle-Größe bleibt klein, weil Tailwind nur genutzte Klassen einbindet.

6.8 Zusammenspiel im Selection-Panel-Projekt

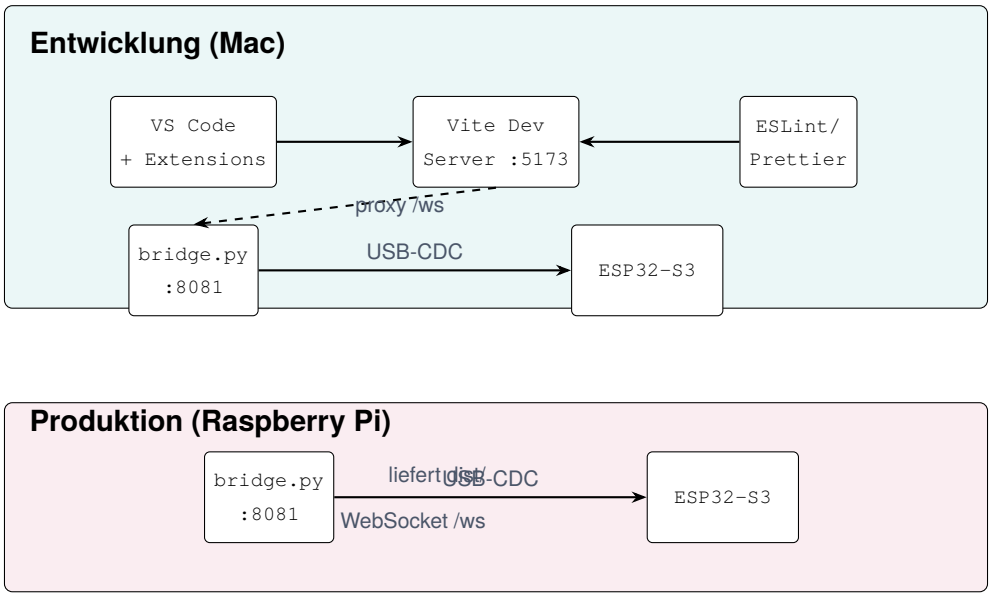


Abbildung 9: Entwicklungs- und Produktionsumgebung

Deployment-Workflow

Listing 24: Frontend-Deployment

```
# Auf dem Mac
cd apps/dashboard
npm run build          # Erzeugt dist/

# Auf den Pi kopieren
rsync -avh --delete \
  ./apps/dashboard/dist/ \
  pi@rover:/home/pi/embedded_projekt/apps/dashboard/dist/
```

6.9 Siehe auch

Weiterführende Dokumentation:

- `apps/dashboard/docs/DEVELOPER.md` – Frontend-Entwicklung
- `apps/dashboard/docs/USER_GUIDE.md` – Benutzerhandbuch
- `apps/bridge/docs/PROTOCOL.md` – WebSocket-Events
- `apps/bridge/docs/WORKFLOW.md` – Pi-Deployment

Anhang

Glossar und Referenzen

A Glossar: Embedded-Entwicklung

Begriffe und Abkürzungen für dieses Projekt (ESP32-S3, CD4021, 74HC595, PlatformIO).

A

Active-Low Signal-Konvention: Aktiv = LOW (0), Inaktiv = HIGH (1).
Im Projekt: CD4021-Buttons sind active-low (gedrückt = 0).

ACK Acknowledge – Bestätigung einer empfangenen Nachricht.
Im Projekt: Bridge sendet ACK nach erfolgreichem Kommando.

B

Baudrate Übertragungsgeschwindigkeit in Bit/s.
Im Projekt: USB-CDC, Baudrate wird ignoriert (921600 nominal).

Bit-Mapping Zuordnung von IDs zu Bit-Positionen.
Im Projekt: MSB-first für Buttons und LEDs.

C

CD4021 8-Bit Parallel-In/Serial-Out Schieberegister.
Im Projekt: Liest Button-Zustände (Active-Low).

COBS Consistent Overhead Byte Stuffing – Framing-Verfahren.
Im Projekt: Für Serial-Protokoll zwischen ESP32 und Pi.

CRC Cyclic Redundancy Check – Prüfsumme für Datenintegrität.
Im Projekt: CRC-16 CCITT für Serial-Frames.

D

Debounce Entprellen von mechanischen Tastern.
Im Projekt: 50 ms Debounce-Zeit, Edge-basiert.

Datenfluss Verarbeitungssequenz: `read` → `update` → `write`
Im Projekt: CD4021 → Logic → HC595

E

Edge Flanke: Übergang von einem Zustand zum anderen.

- **Rising Edge:** $0 \rightarrow 1$
- **Falling Edge:** $1 \rightarrow 0$

Im Projekt: Press-Edge = Taste wird gedrückt.

E2E-Test End-to-End Test mit echter Hardware.
Im Projekt: `test_e2e_hw` prüft SPI, Timing, Mapping.

F

FreeRTOS Real-Time Operating System für Mikrocontroller.
Im Projekt: Optional, für Task-basierte Architektur.

Frame Datenpaket mit Header, Payload, CRC.
Im Projekt: COBS-kodiert über Serial.

G

GPIO General Purpose Input/Output – universelle Pins.

Guard Zeitfenster, das erneute Aktionen blockiert.
Im Projekt: Press-Guard verhindert Doppel-Klicks.

H

HAL Hardware Abstraction Layer – Hardware-Abstraktion.
Im Projekt: `src/hal/` für SPI-Zugriff.

HC595 74HC595: 8-Bit Schieberegister mit Latch.
Im Projekt: Steuert LEDs (Active-High).

I

ISR Interrupt Service Routine – Interrupt-Handler.
Regel: Kurz halten, keine langen Operationen.

L

Latch Speichert Daten für stabile Ausgabe.
Im Projekt: HC595 RCK-Pin triggert Latch.

LSB / MSB
- **LSB:** Least Significant Bit (niedrigstes Bit)
- **MSB:** Most Significant Bit (höchstes Bit)

Im Projekt: MSB-first für Buttons und LEDs.

M

Maske Bitmuster zum Selektieren/Filtern von Bits.

```
value & mask;   // Bits auslesen
value |= mask;  // Bits setzen
```

```
value &= ~mask; // Bits loeschen
```

MISO / MOSI
- **MISO:** Master In, Slave Out (Daten zum Master)
- **MOSI:** Master Out, Slave In (Daten zum Slave)

O

OE Output Enable – Aktiviert Ausgänge (meist active-low).
Im Projekt: HC595 OE = LOW → LEDs aktiv.

One-Hot Genau ein Bit gesetzt (oder keins).
Im Projekt: Immer nur eine LED aktiv.

P

Pipeline Datenfluss durch mehrere Verarbeitungsstufen.
Im Projekt: read → debounce → selection → write

Polling Regelmäßiges Abfragen eines Zustands.
Im Projekt: 10 ms Scan-Intervall.

Pull-up / Pull-down
Widerstand, der Pin auf definierten Pegel zieht.
Im Projekt: Pull-ups für CD4021-Inputs.

R

RTOS Real-Time Operating System.
Im Projekt: Optional FreeRTOS.

S

Scan-Zyklus Ein Durchlauf: Lesen → Verarbeiten → Schreiben.

SPI Serial Peripheral Interface – synchroner Bus.
Im Projekt:
- CD4021: Mode 1, 500 kHz
- HC595: Mode 0, 1 MHz

SSOT Single Source of Truth – eine Quelle für Daten.
Im Projekt: Hardware-Parameter nur in `include/hw/`.

T

Task Ausführungseinheit in FreeRTOS.
Im Projekt: `io_task` für IO-Verarbeitung.

Timing Zeitliche Abfolge von Signalen.
Kritisch: Zu schnell/langsam → Fehler.

U

Unit-Test Test einer einzelnen Komponente.
Im Projekt: Tests für Debounce, Selection, Pipeline.

USB-CDC USB Communication Device Class – virtuelle serielle Schnittstelle.

V

Volatile C-Keyword: Variable kann sich unerwartet ändern (ISR, Hardware).

```
volatile uint8_t flag;
```

W

Watchdog Timer, der bei Timeout einen Reset auslöst.
Zweck: Erkennung von Hängern.

Wire Frame Datenpaket auf dem Draht (nach Encoding).

Weiterführende Dokumentation

`firmware/docs/overview.md` Projektübersicht

`firmware/docs/wiring.md` Hardware-Verdrahtung

`docs/Serial-Protokoll.md` Protokoll-Details